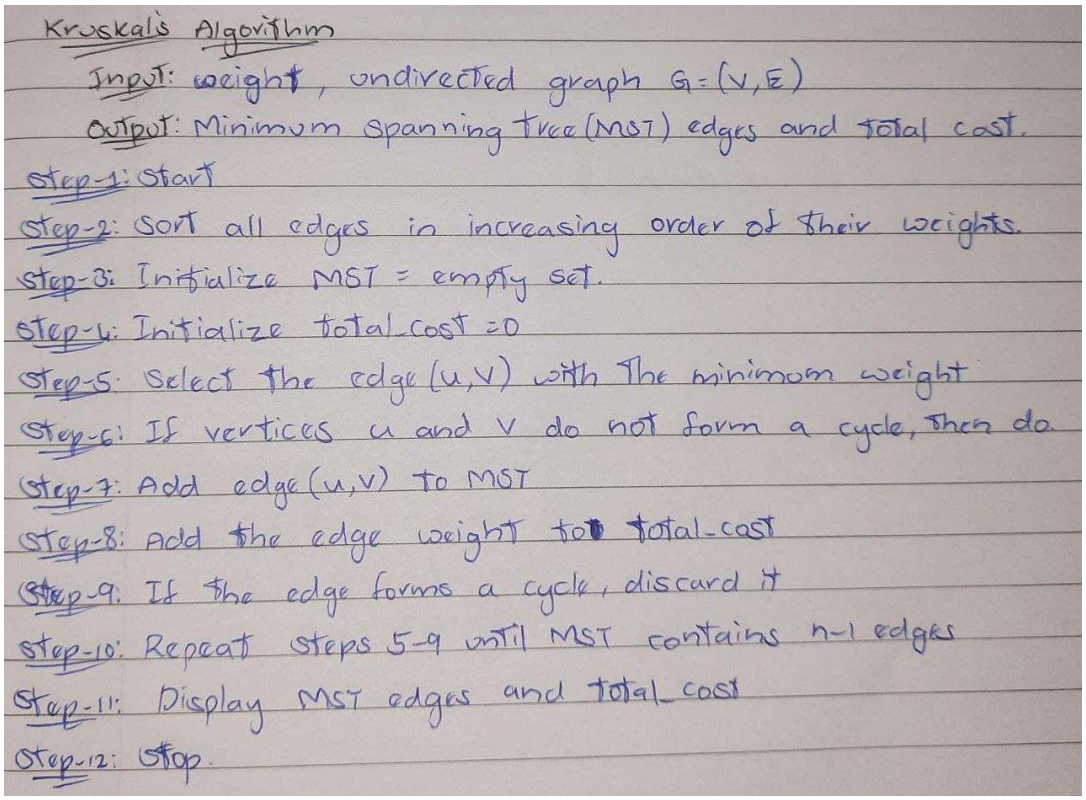
PRACTICAL-10

AIM: Implement Kruskal's algorithm.

ALGORITHM:



PROGRAM:

```
#include <iostream>
#include <algorithm>
using namespace std;
struct Edge {
    int u, v, weight;
};
bool compare(Edge a, Edge b) {
    return a.weight < b.weight;
```

```
}
int parent[20];
int find(int x) {
    if (parent[x] == x)
        return x;
    return parent[x] = find(parent[x]);
}
int main() {
    int n, m;
    Edge e[100];
    cout << "Enter number of vertices: ";
    cin >> n;
    cout << "Enter number of edges: ";
    cin >> m;
    cout << "Enter edges (u v weight):\n";
    for (int i = 0; i < m; i++) {
        cin >> e[i].u >> e[i].v >> e[i].weight;
    }
    sort(e, e + m, compare);

    for (int i = 0; i < n; i++)
        parent[i] = i;
    int total_cost = 0;
    int edge_count = 0;
    cout << "\nEdges in Minimum Spanning Tree:\n";

    for (int i = 0; i < m && edge_count < n - 1; i++) {
        int u = find(e[i].u);
        int v = find(e[i].v);
```

```
        if (u != v) {
            cout << e[i].u << " - " << e[i].v
                 << " : " << e[i].weight << endl;
            parent[u] = v;
            total_cost += e[i].weight;
            edge_count++;
        }
    }
    if (edge_count != n - 1) {
        cout << "Graph is not connected.\n";
    } else {
        cout << "Total cost of MST = " << total_cost << endl;
    }
    return 0;
}
```

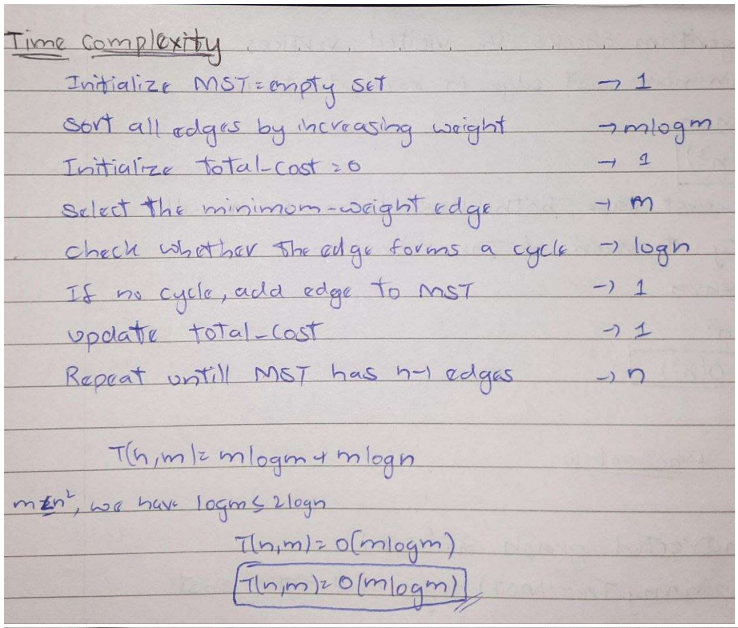
OUTPUT:

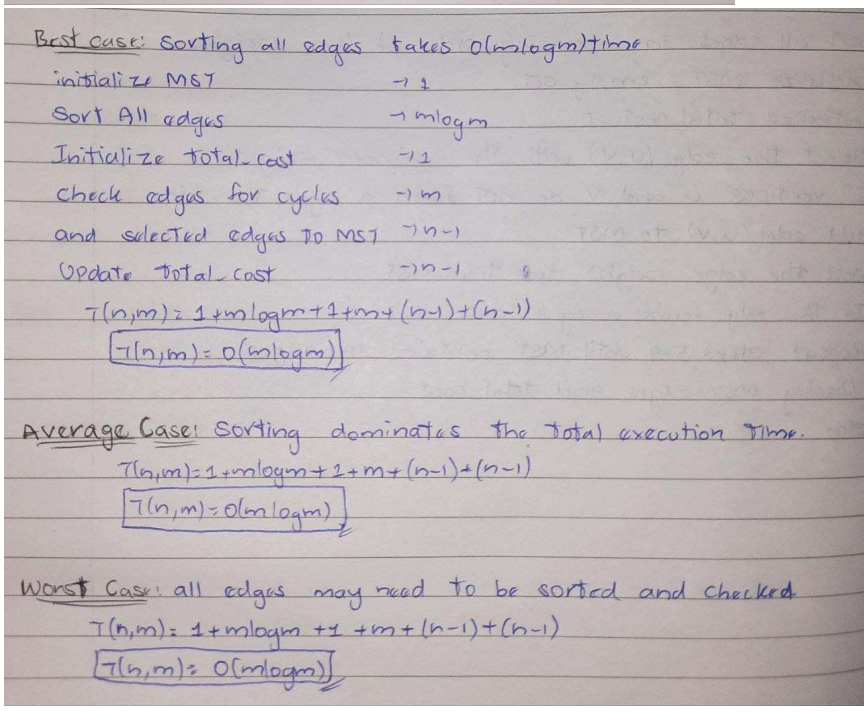
```
Output

Enter number of vertices: 4
Enter number of edges: 5
Enter edges (u v weight):
0 1 2
0 3 6
1 2 3
1 3 8
2 3 1

Edges in Minimum Spanning Tree:
2 - 3 : 1
0 - 1 : 2
1 - 2 : 3
Total cost of MST = 6
```

TIME COMPLEXITY:





CONCLUSION: Kruskal's Algorithm is used to find the Minimum Spanning Tree of a weighted, undirected graph. It selects edges in increasing order of their weights without forming any cycle. It connects all vertices with the minimum total cost. Thus, Kruskal's Algorithm helps to find an efficient way to connect all vertices in a graph.